



Fluxo Computacional Workflow Pipeline: NextFlow

Antonio Mauro Rezende, Ph.D.
Biotecnologia Aplicada ao Estudo de Patógenos
Instituto René Rachou, Fiocruz Minas





Disclaimers:

- O conteúdo dessa apresentação representa a minha opinião. Ele não representa o posicionamento das instituições citadas ao longo da minha fala.
- Apresentação foi criada com o suporte de ferramentas de IA, e eu me responsabilizo por todo conteúdo.



nature biotechnology

[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾

Barcelona - Espanha

[nature](#) > [nature biotechnology](#) > [correspondence](#) > article

[Published: 11 April 2017](#)

Nextflow enables reproducible computational workflows

[Paolo Di Tommaso](#), [Maria Chatzou](#), [Evan W Floden](#), [Pablo Prieto Barja](#), [Emilio Palumbo](#) & [Cedric Notredame](#) 

[Nature Biotechnology](#) **35**, 316–319 (2017) | [Cite this article](#)

16k Accesses | **746** Citations | **121** Altmetric | [Metrics](#)

nextflow

- Linguagem de programação poderosa
- Projetada para o desenvolvimento de pipelines (workflows) de análise de dados escaláveis e reproduzíveis
- Paralelismo natural
- Suporte nativo para geradores de tarefas/agendadores (slurm, PBS, LSF e outros)
- Suporte nativo para Singularity, Docker, Conda e Git (reprodutibilidade e portabilidade)
- Suporte nativo para infraestruturas baseadas em nuvem, como AWS
- Baseada em Groovy

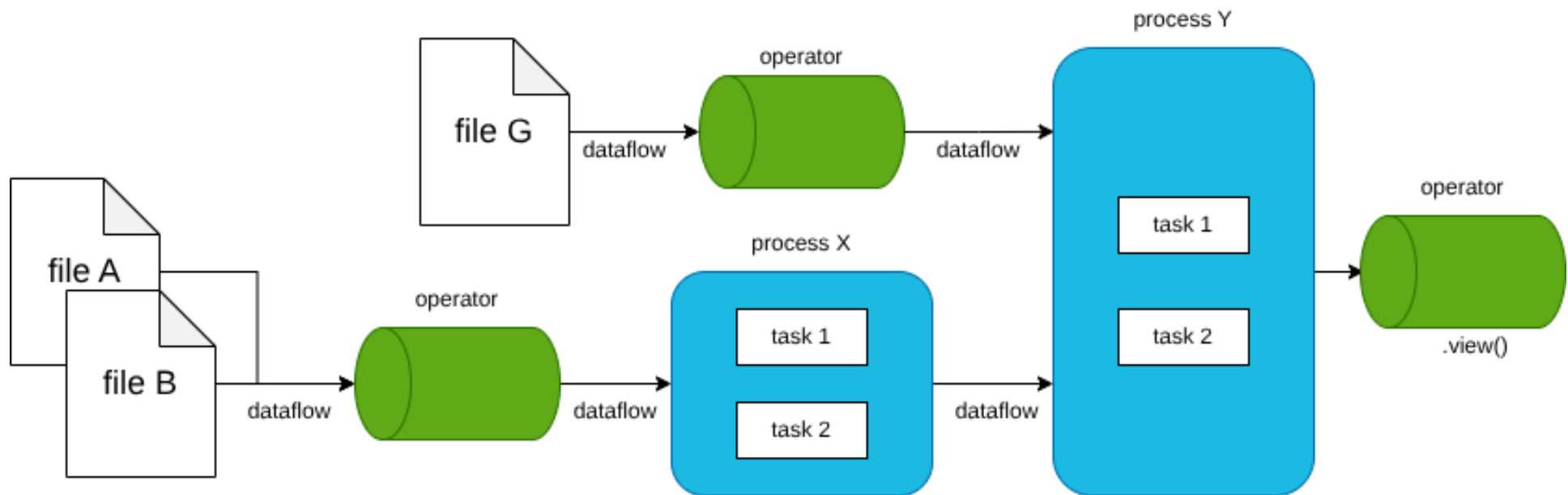




nextflow

- O Nextflow consiste em **quatro** componentes principais:
 - canais, operadores, processos e fluxos de trabalho (workflows).
- Processos: executam um trecho de código (script, ferramenta de linha de comando, etc.) usando a entrada fornecida pelos canais.
- Canais: conectam os processos utilizando fluxos de dados (dataflows).
- Operadores: reorganizam o conteúdo dos canais aplicando funções ou transformações.
- Fluxos de trabalho (Workflows): compõem a lógica do pipeline combinando processos, canais e operadores.

nextflow



Fonte: <https://github.com/vib-training-conferences/nextflow-reproducible-data-analysis>

Caixa de entrada xMail - Antonio Re xDiltheyLab/Meta xTaxonomy brows xTaxonomy brows xStrain-level meta xPhD defence Gab xNF-Core xmodules/.gitpod xnf-core x

nf-co.re

Writing a Simple Ba...How to Access Your...Artic Networkbwa-mem, from .sa...OSF | Centrifuge da...Virosaurus ~ ViralZ...GitHub - jts/sga: de...GitHub - uwb-linux...Creating a NextFlo...Nextflow PatternsSingle-cell RNA-seq...Changing R version...

HomePipelinesModulesToolsDocsEventsAbout

Join nf-core

nf-core



A community effort to collect a curated set of analysis pipelines built using Nextflow.

[VIEW PIPELINES](#)



For facilities

Highly optimised pipelines with excellent reporting. Validated releases ensure reproducibility.



For users

Portable, documented and easy to use workflows. Pipelines that you can trust.



For developers

Companion templates and tools help to validate your code and simplify common tasks.



nf-core is published in Nature Biotechnology!

Pipelines

Browse the 156 pipelines that are currently available as part of nf-core.

Released 97

Under development 46

Archived 13

↓ Last release ▾

☰ ≡

Hint: Scope your search to one field with a prefix: `k:rna` (keyword/topic), `n:sarek` (name), `d:variant` (description), `k:"rna"` (exact keyword/topic).

pixelator ✓

★ 14

New release!

Pipeline to generate Proximity Network Assay data with Pixelator (Pixelgen Technologies AB)

molecular-pixelation pixelator pixelgen-technologies

proteins single-cell single-cell-omics

📦 5.0.1 released about 3 hours ago

seqinspector ✓

★ 28

Dedicated QC-only pipeline for sequencing data. The pipeline will run a (potentially large) set of QC tools and can output global and group specific Multiqc reports. The pipeline is targeting core facilities or research groups with larger sequencing throughput.

genomics quality-control

📦 1.1.2 released 1 day ago

sarek ✓

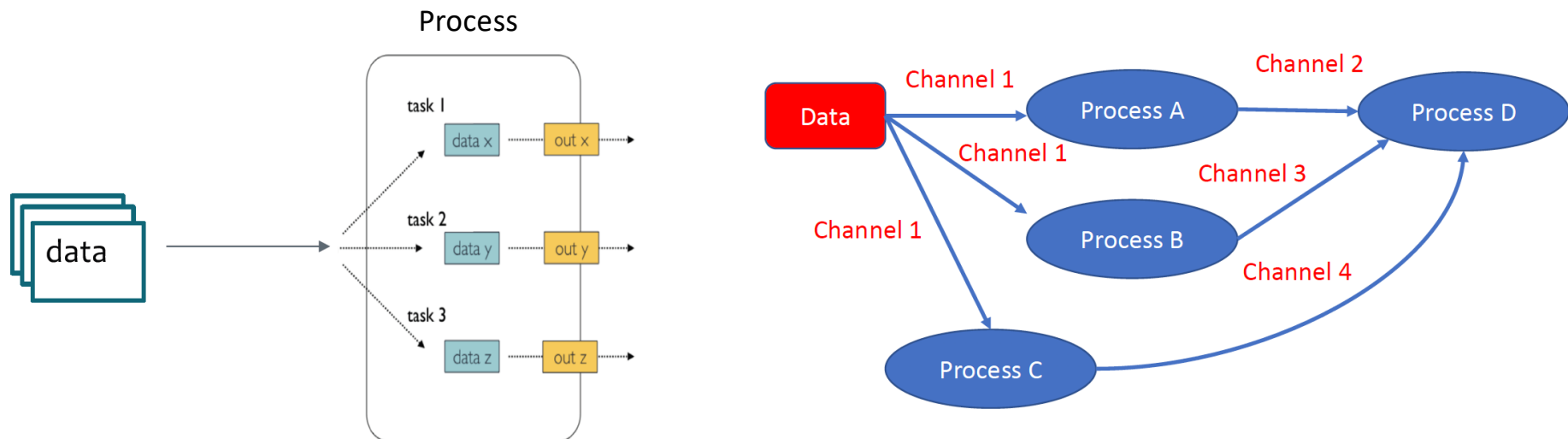
★ 593

Analysis pipeline to detect germline or somatic variants (pre-processing, variant calling and annotation) from WGS / targeted sequencing

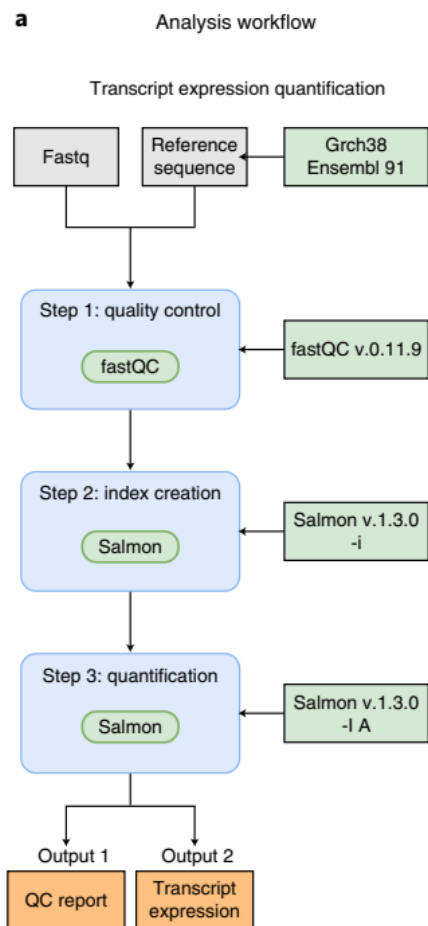
annotation cancer gatk4 genomics germline
pre-processing somatic target-panels variant-calling
whole-exome-sequencing whole-genome-sequencing

📦 3.10.0 released 8 days ago

nextflow

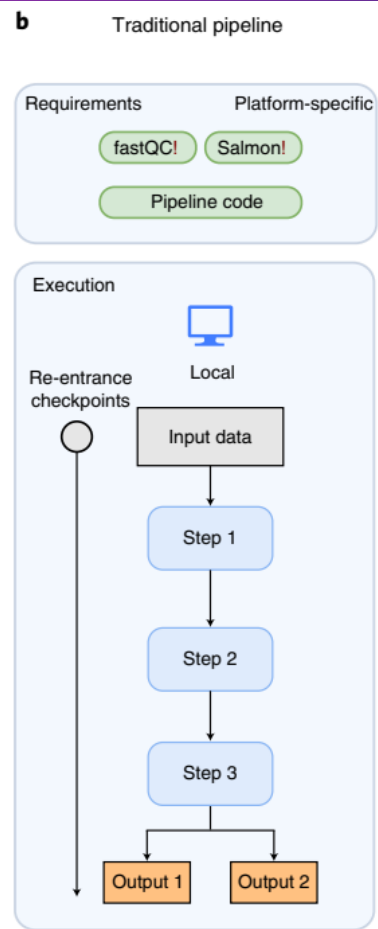


Paralelismo e Reentrância

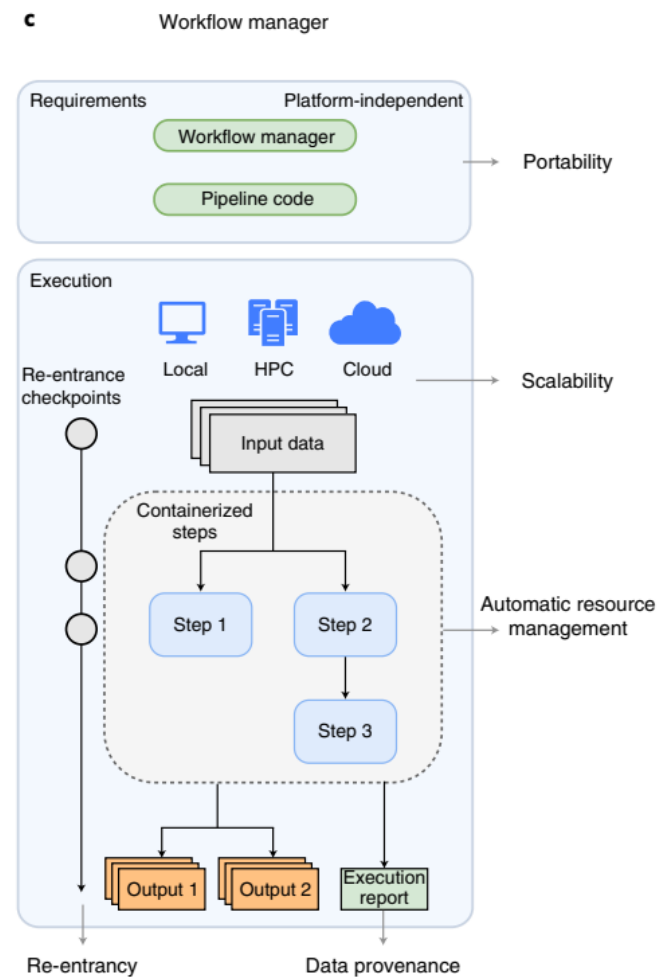


Input data

Output data



Software, versions, parameters



Fixed version, local compute environment

nextflow - Parâmetros, entrada e saída

```
hello-nextflow >  hello-world.nf
1  #!/usr/bin/env nextflow
2
3  /*
4   * Use echo to print 'Hello World!' to a file
5   */
6  process sayHello {
7
8      output:
9      path 'output.txt'
10
11     script:
12     """
13     echo 'Hello World!' > output.txt
14     """
15 }
16
17 workflow {
18
19     main:
20     // emit a greeting
21     sayHello()
22 }
23
```

- `training/hello-nextflow -> nextflow run hello-world.nf`
Nextflow 26.04.6 is available - Please consider updating your version to it

N E X T F L O W ~ version 26.04.4

Launching `hello-world.nf` [**disturbed_morse**] revision: dc38bebf45

executor > local (1)
[cb/0bc678] sayHello [100%] 1 of 1 ✓

- `training/hello-nextflow -> █`

- `training/hello-nextflow/work/cb/0bc678d2504cd6466ab1224f83f24e -> pwd`
/workspaces/training/hello-nextflow/work/cb/0bc678d2504cd6466ab1224f83f24e
- `training/hello-nextflow/work/cb/0bc678d2504cd6466ab1224f83f24e -> ls`
output.txt
- `training/hello-nextflow/work/cb/0bc678d2504cd6466ab1224f83f24e -> more output.txt`
Hello World!

- `training/hello-nextflow/work/cb/0bc678d2504cd6466ab1224f83f24e -> ls -laht`
total 24K
drwxrwxrwx+ 2 root root 4.0K Aug 20 19:14 
-rw-rw-rw- 1 root root 1 Aug 20 19:14 .exitcode
-rw-rw-rw- 1 root root 0 Aug 20 19:14 .command.err
-rw-rw-rw- 1 root root 0 Aug 20 19:14 .command.out
-rw-rw-rw- 1 root root 13 Aug 20 19:14 output.txt
-rw-rw-rw- 1 root root 0 Aug 20 19:14 .command.begin
-rw-rw-rw- 1 root root 0 Aug 20 19:14 .command.log
-rw-rw-rw- 1 root root 3.1K Aug 20 19:14 .command.run
-rw-rw-rw- 1 root root 49 Aug 20 19:14 .command.sh
drwxrwxrwx+ 3 root root 4.0K Aug 20 19:14 

nextflow - Parâmetros, entrada e saída

```
hello-nextflow > solutions > 1-hello-world >  hello-world-2.nf
```

```
1  /workspaces/training/hello-nextflow
2
3  /*
4   * Use echo to print 'Hello World!' to a file
5   */
6  process sayHello {
7
8      output:
9      path 'output.txt'
10
11      script:
12      """
13      echo 'Hello World!' > output.txt
14      """
15  }
16
17  workflow {
18
19      main:
20      // emit a greeting
21      sayHello()
22
23      publish:
24      first_output = sayHello.out
25  }
26
27  output {
28      first_output {
29          path 'hello_world'
30          //mode 'copy'
31      }
32  }
33
```

```
● training/hello-nextflow/solutions/1-hello-world/results/hello_world -> pwd
/workspaces/training/hello-nextflow/solutions/1-hello-world/results/hello_world
● training/hello-nextflow/solutions/1-hello-world/results/hello_world -> ls -lah
total 12K
drwxrwxrwx+ 2 root root 4.0K Aug 20 19:22 
drwxrwxrwx+ 3 root root 4.0K Aug 20 19:22 
lrwxrwxrwx 1 root root 109 Aug 20 19:22 output.txt -> /workspaces/training/hello-nextflow/solutions/1-hello-world/work/5e/f
ed66ea911eb384641514f3e6b201e/output.txt
```

```
● training/hello-nextflow/solutions/1-hello-world/results/hello_world -> pwd
/workspaces/training/hello-nextflow/solutions/1-hello-world/results/hello_world
● training/hello-nextflow/solutions/1-hello-world/results/hello_world -> ls -lah
total 12K
drwxrwxrwx+ 2 root root 4.0K Aug 20 19:24 
drwxrwxrwx+ 3 root root 4.0K Aug 20 19:22 
-rw-rw-rw- 1 root root 13 Aug 20 19:24 output.txt
```

nextflow - Parâmetros, entrada e saída

```
hello-nextflow > solutions > 1-hello-world > X hello-world-3.nf
1  #!/usr/bin/env nextflow
2
3  /*
4   * Use echo to print 'Hello World!' to a file
5   */
6  process sayHello {
7
8      input:
9      val greeting
10
11      output:
12      path 'output.txt'
13
14      script:
15      """
16      echo '${greeting}' > output.txt
17      """
18  }
19
20  /*
21   * Pipeline parameters
22   */
23  params {
24      input: String = 'Hola mundo!'
25  }
26
27  workflow {
28
29      main:
30      // emit a greeting
31      sayHello(params.input)
32
33      publish:
34      first_output = sayHello.out
35  }
36
37  output {
38      first_output {
39          path 'hello_world'
40          mode 'copy'
41      }
42  }
43
```

- `training/hello-nextflow/solutions/1-hello-world -> ls`
hello-world-2.nf hello-world-3.nf **results work**
- `training/hello-nextflow/solutions/1-hello-world -> nextflow run hello-world-3.nf --input "ola teste"`
Nextflow 26.04.6 is available – Please consider updating your version to it

NEXTFLOW ~ version 26.04.4

Launching `hello-world-3.nf` [**tender_murdock**] revision: 8336e5ce9e

executor > local (1)
[c5/ea9f12] sayHello [100%] 1 of 1 ✓

Outputs:

/workspaces/training/hello-nextflow/solutions/1-hello-world/results

first_output: hello_world/output.txt

- `training/hello-nextflow/solutions/1-hello-world -> ls`
hello-world-2.nf hello-world-3.nf **results work**
- `training/hello-nextflow/solutions/1-hello-world -> cd results/`
- `training/hello-nextflow/solutions/1-hello-world/results -> ls`
hello_world
- `training/hello-nextflow/solutions/1-hello-world/results -> cd hello_world/`
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> ls`
output.txt
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> more output.txt`
ola teste
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> █`

nextflow - Parâmetros, entrada e saída

```
hello-nextflow > solutions > 1-hello-world > X hello-world-3.nf
1  #!/usr/bin/env nextflow
2
3  /*
4   * Use echo to print 'Hello World!' to a file
5   */
6  process sayHello {
7
8      input:
9      val greeting
10
11      output:
12      path 'output.txt'
13
14      script:
15      """
16      echo '${greeting}' > output.txt
17      """
18  }
19
20  /*
21   * Pipeline parameters
22   */
23  params {
24      input: String = 'Hola mundo!'
25  }
26
27  workflow {
28
29      main:
30      // emit a greeting
31      sayHello(params.input)
32
33      publish:
34      first_output = sayHello.out
35  }
36
37  output {
38      first_output {
39          path 'hello_world'
40          mode 'copy'
41      }
42  }
43
```

- `training/hello-nextflow/solutions/1-hello-world -> ls`
hello-world-2.nf hello-world-3.nf **results work**
- `training/hello-nextflow/solutions/1-hello-world -> nextflow run hello-world-3.nf --input "ola teste"`
Nextflow 26.04.6 is available – Please consider updating your version to it

NEXTFLOW ~ version 26.04.4

Launching `hello-world-3.nf` [**tender_murdock**] revision: 8336e5ce9e

executor > local (1)
[c5/ea9f12] sayHello [100%] 1 of 1 ✓

Outputs:

/workspaces/training/hello-nextflow/solutions/1-hello-world/results

first_output: hello_world/output.txt

- `training/hello-nextflow/solutions/1-hello-world -> ls`
hello-world-2.nf hello-world-3.nf **results work**
- `training/hello-nextflow/solutions/1-hello-world -> cd results/`
- `training/hello-nextflow/solutions/1-hello-world/results -> ls`
hello_world
- `training/hello-nextflow/solutions/1-hello-world/results -> cd hello_world/`
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> ls`
output.txt
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> more output.txt`
ola teste
- `training/hello-nextflow/solutions/1-hello-world/results/hello_world -> |`

nextflow - Canais e Operadores

```
workflow {  
  
    main:  
    // create a channel for inputs  
    greeting_ch = channel.of('Hello Channels!')  
    | | | | |.view()  
    // emit a greeting  
    sayHello(greeting_ch)  
  
    publish:  
    first_output = sayHello.out  
}
```

```
workflow {  
  
    main:  
    // create a channel for inputs  
    greeting_ch = channel.of('Hello', 'Bonjour', 'Hola')  
    | | | | |.view()  
    // emit a greeting  
    sayHello(greeting_ch)  
  
    publish:  
    first_output = sayHello.out  
}
```

```
training/hello-nextflow/solutions/2-hello-channels -> nextflow run hello-channels-2.nf  
Nextflow 26.04.6 is available - Please consider updating your version to it  
  
NEXTFLOW ~ version 26.04.4  
Launching 'hello-channels-2.nf' [jovial_sax] revision: 6590203031  
  
executor > local (3)  
[7c/b18040] sayHello (2) | 3 of 3 ✓  
Hello  
Bonjour  
Hola  
  
Outputs:  
  
/workspaces/training/hello-nextflow/solutions/2-hello-channels/results  
  
first_output:  
- hello_channels/Hello-output.txt  
- hello_channels/Bonjour-output.txt  
- hello_channels/Hola-output.txt
```

```
process sayHello {  
  
    input:  
    val greeting  
  
    output:  
    path "${greeting}-output.txt"  
  
    script:  
    """"  
    echo '${greeting}' > '${greeting}-output.txt'  
    """"  
}
```

nextflow - Canais e Operadores

```
workflow {  
  
    main:  
    // declare an array of input greetings  
    greetings_array = ['Hello','Bonjour','Hola']  
    // create a channel for inputs  
    greeting_ch = channel.of(greetings_array)  
    | | | | |  
    .view { greeting -> "Before flatten: $greeting" }  
    .flatten()  
    .view { greeting -> "After flatten: $greeting" }  
    // emit a greeting  
    sayHello(greeting_ch)  
  
    publish:  
    first_output = sayHello.out  
}
```

- Erro sem o operador flatten
- Array processador de uma vez sem o operador

nextflow - Canais e Operadores

```
hello-channels-4.nf × greetings.csv ×
hello-nextflow > data > greetings.csv
1 Hello,English,123
2 Bonjour,French,456
3 Hola,Spanish,789
```

Arquivo de entrada – caso mais comum em bioinfo

- Poderia usar o flatten, mas teria 9 execuções
- Quero somente o primeira coluna do arquivo
- operador map é genérico para construir mudanças no canal

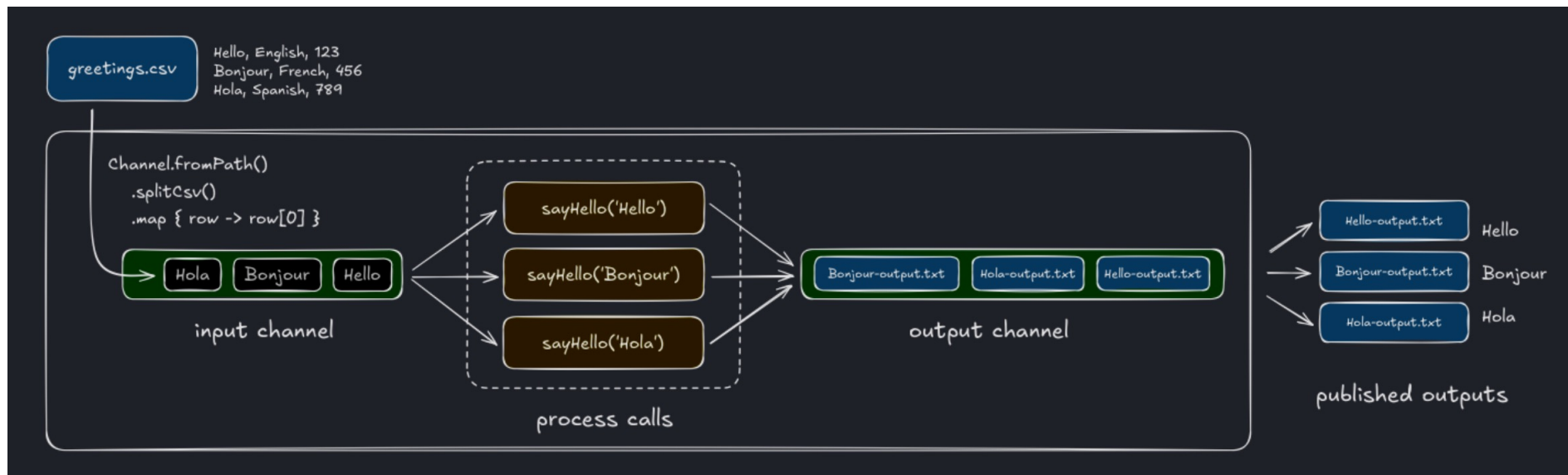
```
/*
 * Pipeline parameters
 */
params {
    input: Path = 'data/greetings.csv'
}

workflow {
    main:
    // create a channel for inputs from a CSV file
    greeting_ch = channel.fromPath(params.input)
    .view { csv -> "Before splitCsv: $csv" }
    .splitCsv()
    .view { csv -> "After splitCsv: $csv" }
    .map { item -> item[0] }
    .view { csv -> "After map: $csv" }
    // emit a greeting
    sayHello(greeting_ch)

    publish:
    first_output = sayHello.out
}
```

nextflow - workflow

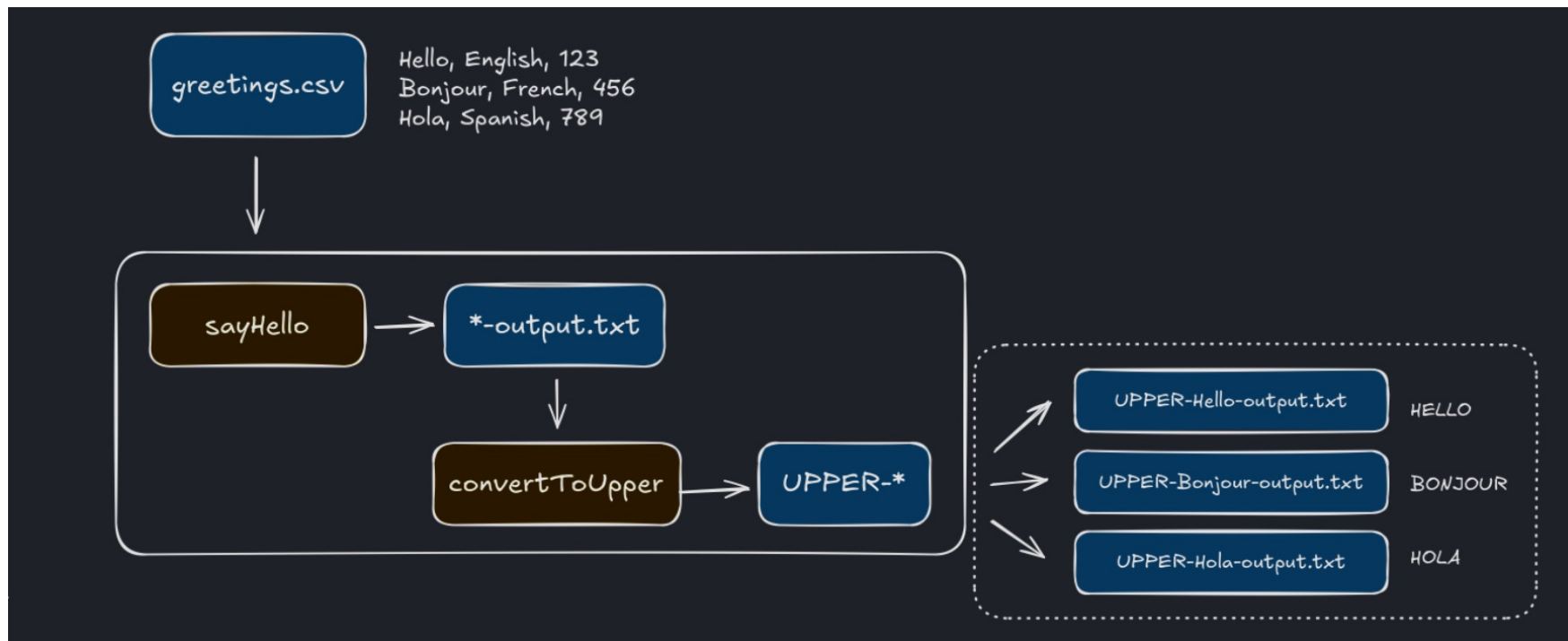
O QUE O CÓDIGO ESTÁ FAZENDO ATUALMENTE:



Fonte: https://training.nextflow.io/latest/pt/hello_nextflow/03_hello_workflow/#1-adicione-uma-segunda-etapa-ao-fluxo-de-trabalho

nextflow - workflow

O QUE EU QUERO QUE ELE FAÇA:



Fonte: https://training.nextflow.io/latest/pt/hello_nextflow/03_hello_workflow/#1-adicione-uma-segunda-etapa-ao-fluxo-de-trabalho

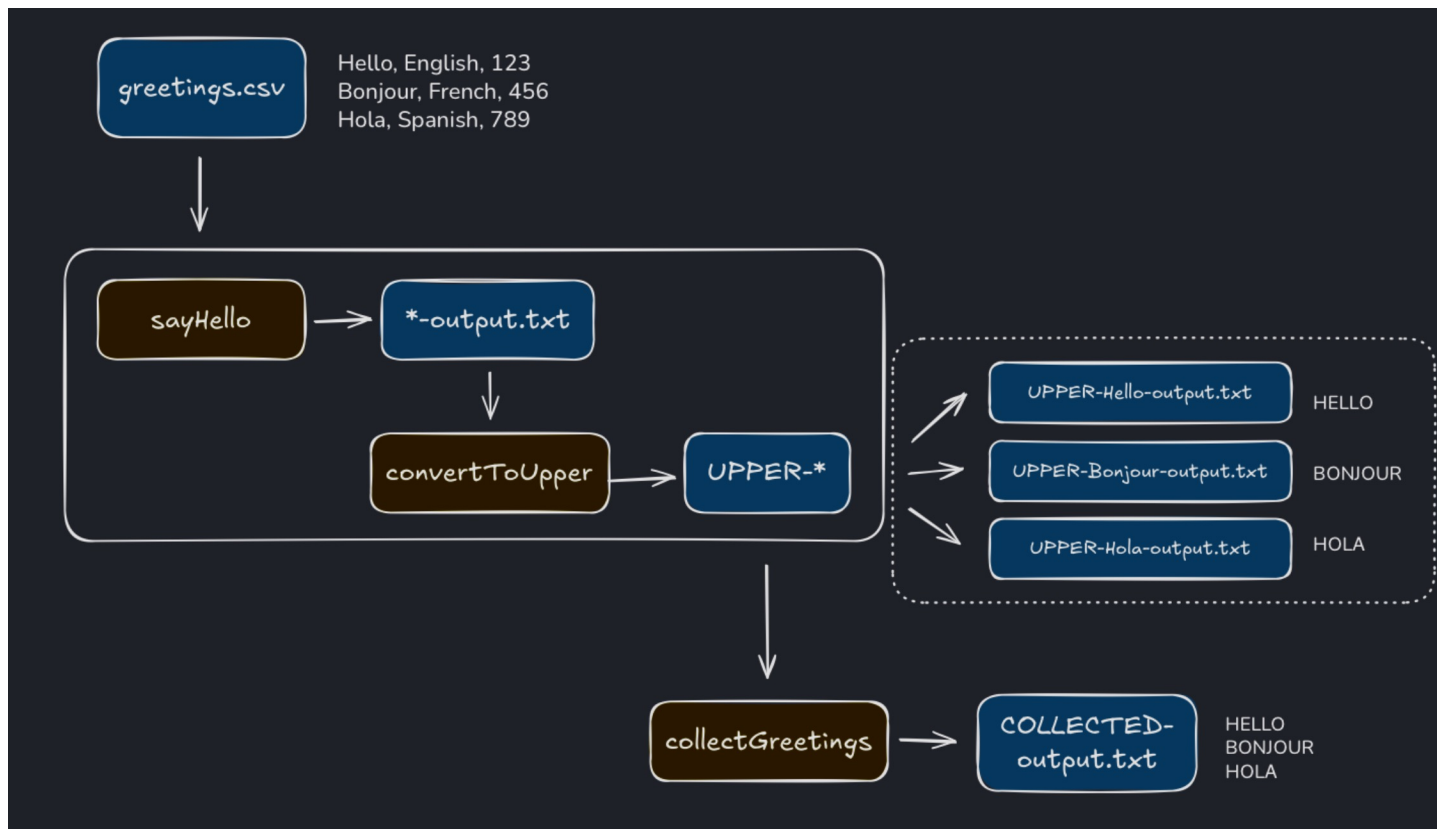
nextflow - workflow

```
process sayHello {  
  
    input:  
    val greeting  
  
    output:  
    path "${greeting}-output.txt"  
  
    script:  
    """  
    echo '${greeting}' > '${greeting}-output.txt'  
    """  
}
```

```
process convertToUpper {  
  
    input:  
    path input_file  
  
    output:  
    path "UPPER-${input_file}"  
  
    script:  
    """  
    cat '${input_file}' | tr 'a-z' 'A-Z' > 'UPPER-${input_file}'  
    """  
}
```

```
workflow {  
  
    main:  
    // create a channel for inputs from a CSV file  
    greeting_ch = channel.fromPath(params.input)  
                    .splitCsv()  
                    .map { line -> line[0] }  
    // emit a greeting  
    sayHello(greeting_ch)  
    // convert the greeting to uppercase  
    convertToUpper(sayHello.out)  
  
    publish:  
    first_output = sayHello.out  
    uppercased = convertToUpper.out  
}  
  
output {  
    first_output {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    uppercased {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
}
```

nextflow - workflow



Fonte: https://training.nextflow.io/latest/pt/hello_nextflow/03_hello_workflow/#1-adicione-uma-segunda-etapa-ao-fluxo-de-trabalho



nextflow - workflow

```
echo 'Hello' | tr '[a-z]' '[A-Z]' > UPPER-Hello-output.txt
```

```
echo 'Bonjour' | tr '[a-z]' '[A-Z]' > UPPER-Bonjour-output.txt
```

```
echo 'Hola' | tr '[a-z]' '[A-Z]' > UPPER-Hola-output.txt
```

```
cat UPPER-Hello-output.txt UPPER-Bonjour-output.txt UPPER-Hola-output.txt > COLLECTED-output.txt
```

nextflow - workflow

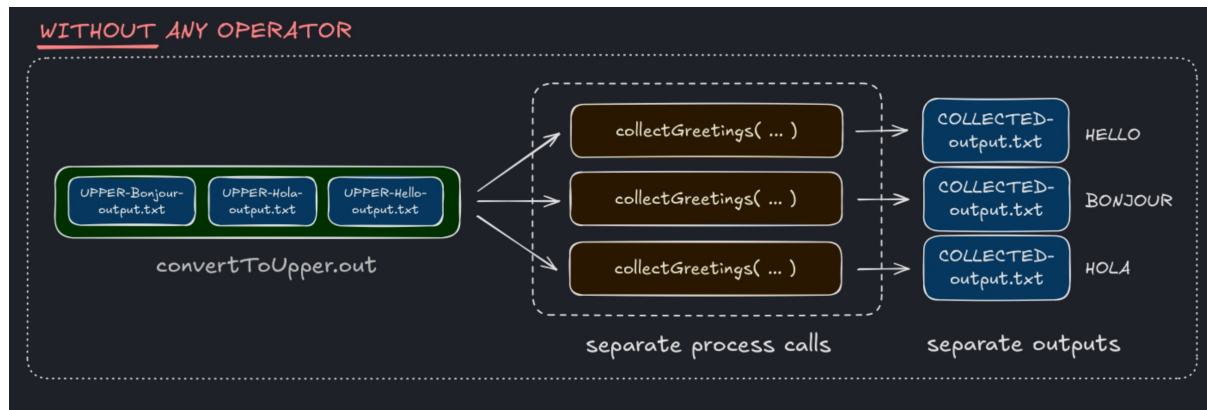
```
process sayHello {  
  
    input:  
    val greeting  
  
    output:  
    path "${greeting}-output.txt"  
  
    script:  
    """"  
    echo '${greeting}' > '${greeting}-output.txt'  
    """"  
}
```

```
process convertToUpper {  
  
    input:  
    path input_file  
  
    output:  
    path "UPPER-${input_file}"  
  
    script:  
    """"  
    cat '${input_file}' | tr 'a-z' 'A-Z' > 'UPPER-${input_file}'  
    """"  
}
```

```
process collectGreetings {  
  
    input:  
    path input_files  
  
    output:  
    path "COLLECTED-output.txt"  
  
    script:  
    """"  
    cat ${input_files} > 'COLLECTED-output.txt'  
    """"  
}
```

```
workflow {  
  
    main:  
    // create a channel for inputs from a CSV file  
    greeting_ch = channel.fromPath(params.input)  
                    .splitCsv()  
                    .map { line -> line[0] }  
  
    // emit a greeting  
    sayHello(greeting_ch)  
    // convert the greeting to uppercase  
    convertToUpper(sayHello.out)  
    // collect all the greetings into one file  
    collectGreetings(convertToUpper.out.collect())  
  
    // optional view statements  
    convertToUpper.out.view { item -> "Before collect: ${item}" }  
    convertToUpper.out.collect().view { items -> "After collect: ${items}" }  
  
    publish:  
    first_output = sayHello.out  
    uppercased = convertToUpper.out  
    collected = collectGreetings.out  
}  
  
output {  
    first_output {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    uppercased {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    collected {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
}
```

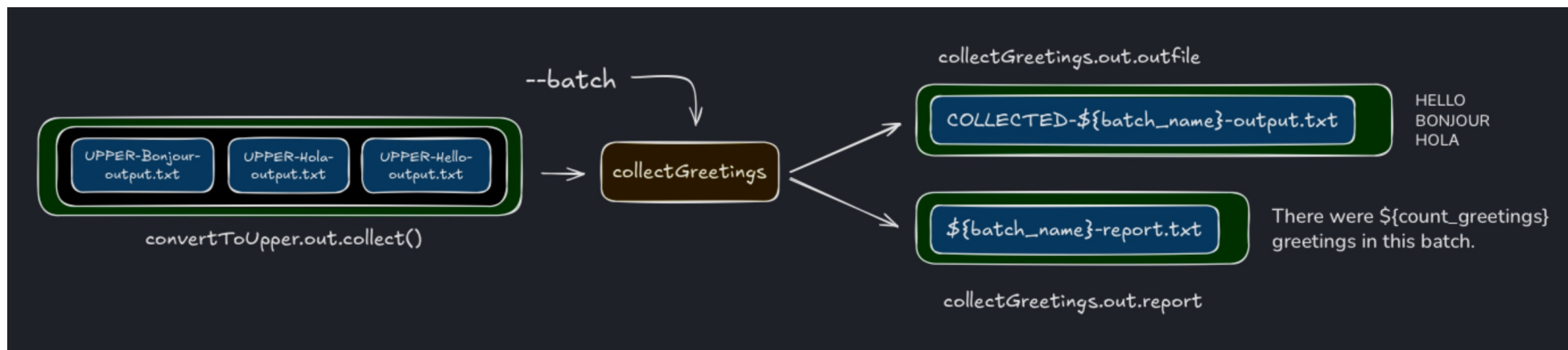
nextflow - workflow



```
workflow {  
  
    main:  
    // create a channel for inputs from a CSV file  
    greeting_ch = channel.fromPath(params.input)  
                  .splitCsv()  
                  .map { line -> line[0] }  
  
    // emit a greeting  
    sayHello(greeting_ch)  
    // convert the greeting to uppercase  
    convertToUpper(sayHello.out)  
    // collect all the greetings into one file  
    collectGreetings(convertToUpper.out.collect())  
  
    // optional view statements  
    convertToUpper.out.view { item -> "Before collect: ${item}" }  
    convertToUpper.out.collect().view { items -> "After collect: ${items}" }  
  
    publish:  
    first_output = sayHello.out  
    uppercased = convertToUpper.out  
    collected = collectGreetings.out  
}  
  
output {  
    first_output {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    uppercased {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    collected {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
}
```

nextflow - workflow

Processo com mais de uma entrada e mais de um tipo de saída



nextflow - workflow

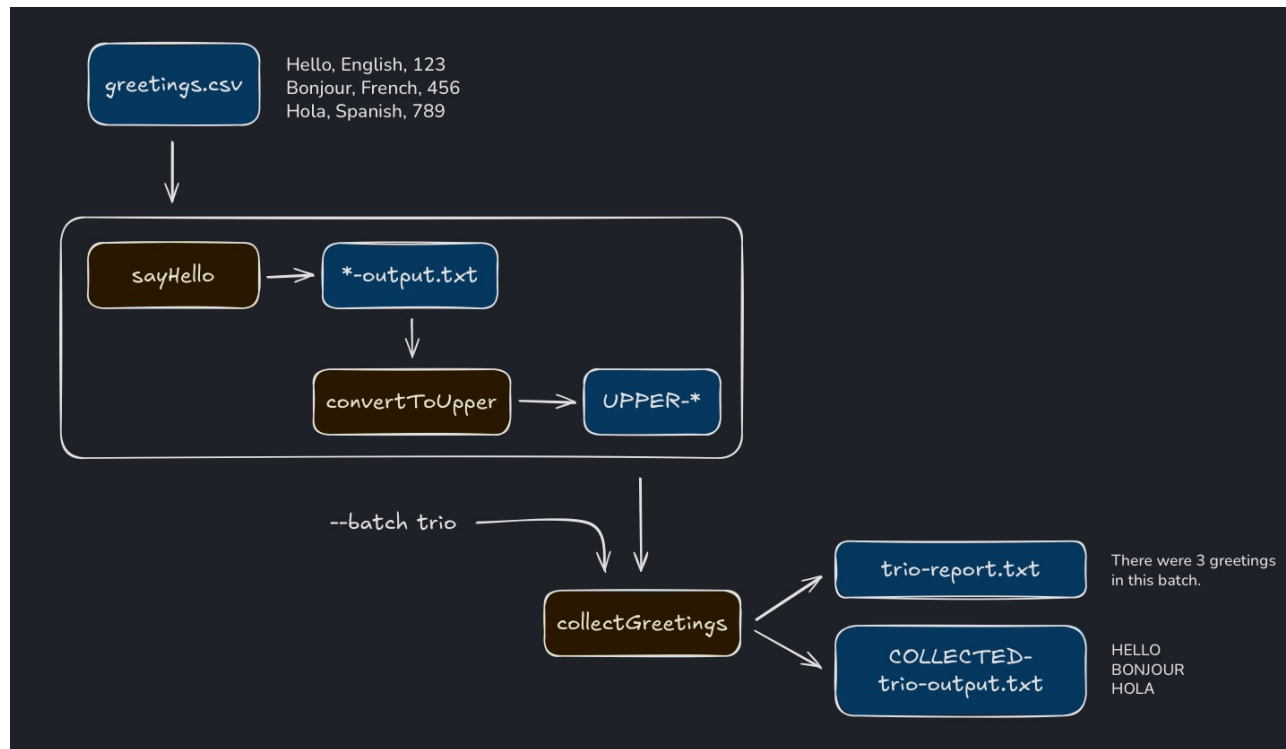
```
process collectGreetings {  
  
    input:  
    path input_files  
    val batch_name  
  
    output:  
    path "COLLECTED-${batch_name}-output.txt", emit: outfile  
    path "${batch_name}-report.txt", emit: report  
  
    script:  
    count_greetings = input_files.size()  
    """"  
    cat ${input_files} > 'COLLECTED-${batch_name}-output.txt'  
    echo 'There were ${count_greetings} greetings in this batch.' > '${batch_name}-report.txt'  
    """"  
}
```

```
/*  
 * Pipeline parameters  
 */  
params {  
    input: Path = 'data/greetings.csv'  
    batch: String = 'batch'  
}  
  
Preview DAG  
workflow {  
  
    main:  
    // create a channel for inputs from a CSV file  
    greeting_ch = channel.fromPath(params.input)  
                    .splitCsv()  
                    .map { line -> line[0] }  
  
    // emit a greeting  
    sayHello(greeting_ch)  
    // convert the greeting to uppercase  
    convertToUpper(sayHello.out)  
    // collect all the greetings into one file  
    collectGreetings(convertToUpper.out.collect(), params.batch)  
  
    publish:  
    first_output = sayHello.out  
    uppercased = convertToUpper.out  
    collected = collectGreetings.out.outfile  
    batch_report = collectGreetings.out.report  
}
```

```
output {  
    first_output {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    uppercased {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    collected {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
    batch_report {  
        path 'hello_workflow'  
        mode 'copy'  
    }  
}
```

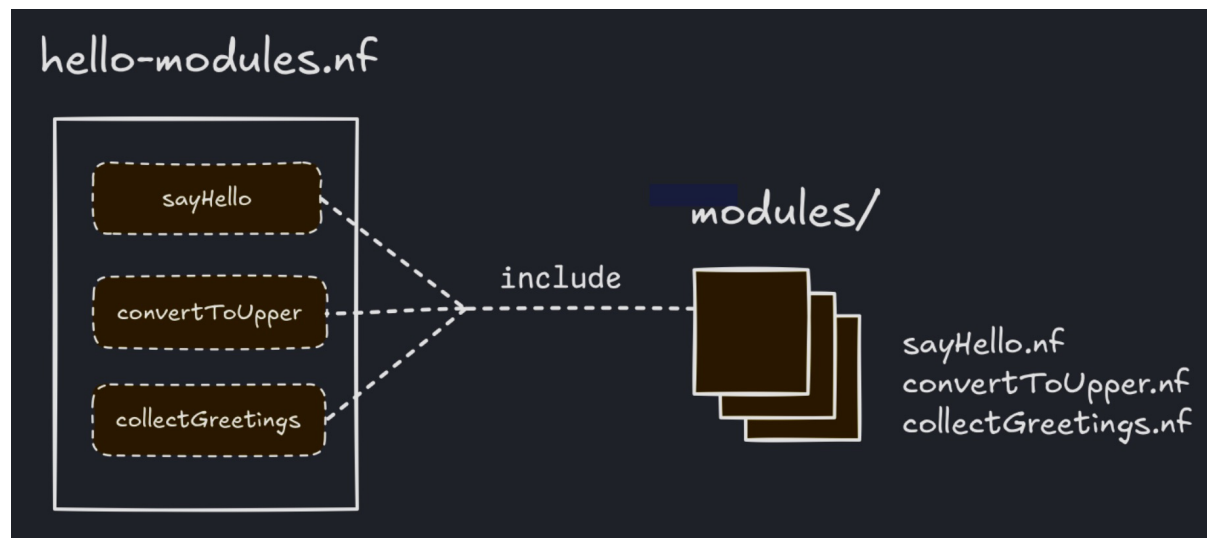
nextflow - workflow

No final temos esse pipeline!



nextflow - Módulos

- Até o momento, todos os processos dentro de um único código
- Código do pipeline cresce = dificuldade de manutenção e de compartilhamento
- Maior facilidade de navegar no código



Fonte: https://training.nextflow.io/latest/hello_nextflow/04_hello_modules/

nextflow - Módulos

```
1  #!/usr/bin/env nextflow
2
3  // Include modules
4  include { sayHello } from './modules/sayHello.nf'
5  include { convertToUpper } from './modules/convertToUpper.nf'
6  include { collectGreetings } from './modules/collectGreetings.nf'
7
8  /*
9   * Pipeline parameters
10  */
11  params {
12      input: Path = 'data/greetings.csv'
13      batch: String = 'batch'
14  }
15
16  Preview DAG
17  workflow {
18      main:
19      // create a channel for inputs from a CSV file
20      greeting_ch = channel.fromPath(params.input)
21                      .splitCsv()
22                      .map { line -> line[0] }
23      // emit a greeting
24      sayHello(greeting_ch)
25      // convert the greeting to uppercase
26      convertToUpper(sayHello.out)
27      // collect all the greetings into one file
28      collectGreetings(convertToUpper.out.collect(), params.batch)
29  }
```

```
29
30  publish:
31  first_output = sayHello.out
32  uppercased = convertToUpper.out
33  collected = collectGreetings.out.outfile
34  batch_report = collectGreetings.out.report
35  }
36
37  output {
38      first_output {
39          path 'hello_modules'
40          mode 'copy'
41      }
42      uppercased {
43          path 'hello_modules'
44          mode 'copy'
45      }
46      collected {
47          path 'hello_modules'
48          mode 'copy'
49      }
50      batch_report {
51          path 'hello_modules'
52          mode 'copy'
53      }
54  }
55
```



nextflow - Containers - Docker

- **Imagem = Todos os arquivos e sistema**
- **Containers = A instância onde se trabalha**
- **Container é criado a partir de uma imagem**
- **Totalmente isolado do resto do sistema**
- **Isso garante reprodutibilidade - mesma versão de tudo (biblioteca, etc..)**

nextflow - Containers - Docker

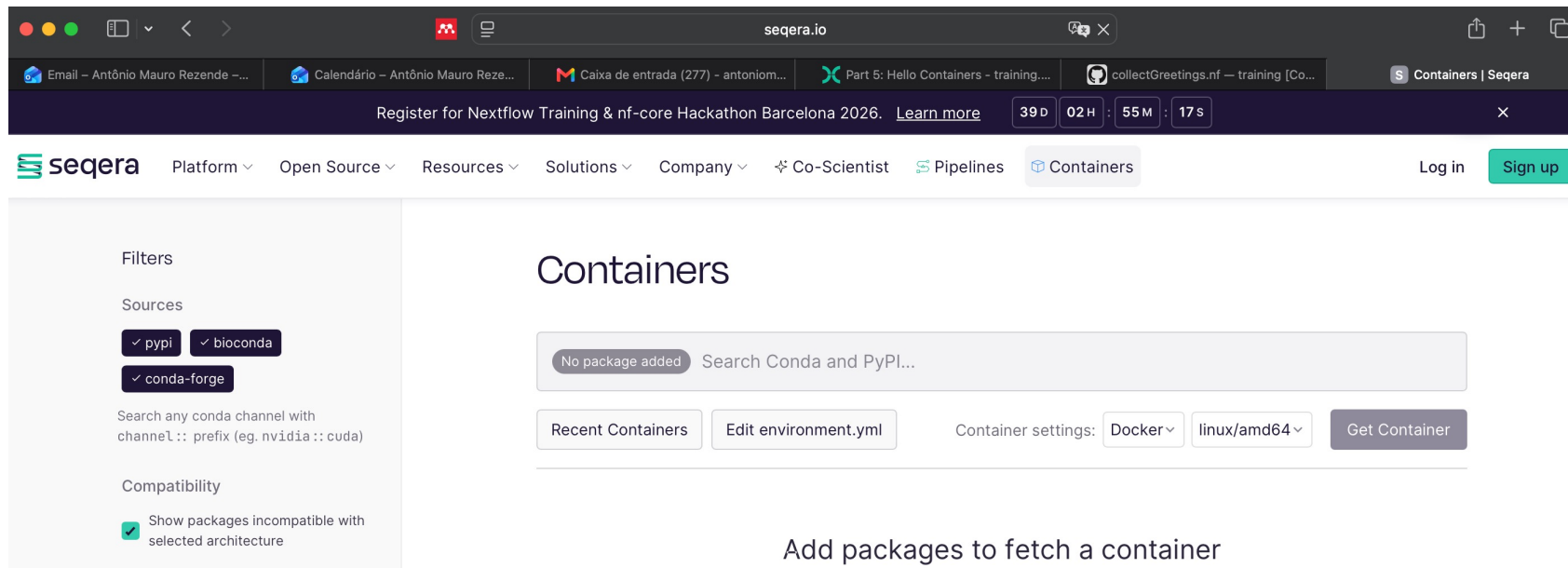
- `training/hello-nextflow/solutions/4-hello-modules` -> `docker -v`
Docker version 29.5.3-1, build d1c06ef6b41d88d76866aea43c246cd7c63d04fa
- `training/hello-nextflow/solutions/4-hello-modules` -> `docker image ls`

IMAGE	ID	DISK USAGE	Info → CONTENT	U SIZE	In Use EXTRA
ghcr.io/cs50/codespace:latest	8ca15bd6372f	3.59GB		0B	
ghcr.io/nextflow-io/training:latest	312825c774e3	2.73GB		0B	
mcr.microsoft.com/devcontainers/base:bookworm	effb943e85cd	843MB		0B	
mcr.microsoft.com/devcontainers/base:debian	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/devcontainers/base:debian12	effb943e85cd	843MB		0B	
mcr.microsoft.com/devcontainers/base:debian13	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/devcontainers/base:latest	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/devcontainers/base:trixie	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/devcontainers/universal:5	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/devcontainers/universal:5-linux	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/devcontainers/universal:5-noble	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/devcontainers/universal:latest	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/vscode/devcontainers/base:bookworm	effb943e85cd	843MB		0B	
mcr.microsoft.com/vscode/devcontainers/base:debian	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/vscode/devcontainers/base:debian12	effb943e85cd	843MB		0B	
mcr.microsoft.com/vscode/devcontainers/base:debian13	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/vscode/devcontainers/base:latest	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/vscode/devcontainers/base:trixie	cc9cb1c81e52	921MB		0B	
mcr.microsoft.com/vscode/devcontainers/universal:5	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/vscode/devcontainers/universal:5-linux	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/vscode/devcontainers/universal:5-noble	d38910706dfe	9.74GB		0B	
mcr.microsoft.com/vscode/devcontainers/universal:latest	d38910706dfe	9.74GB		0B	

- `training/hello-nextflow/solutions/4-hello-modules` -> █

nextflow - Containers - Docker

Imagem da Internet - Seqera



The screenshot shows the Seqera.io website interface. At the top, there's a navigation bar with the Seqera logo and links for Platform, Open Source, Resources, Solutions, Company, Co-Scientist, Pipelines, and Containers. A 'Log in' button and a 'Sign up' button are also present. Below the navigation bar, there's a sidebar on the left with 'Filters' and 'Sources' sections. The 'Sources' section has buttons for 'pypi', 'bioconda', and 'conda-forge'. The main content area is titled 'Containers' and features a search bar with the placeholder text 'No package added Search Conda and PyPI...'. Below the search bar, there are buttons for 'Recent Containers', 'Edit environment.yml', and 'Container settings:'. The 'Container settings' section shows 'Docker' and 'linux/amd64' as selected options, with a 'Get Container' button. At the bottom of the page, there's a text prompt: 'Add packages to fetch a container'.

Filters

Sources

✓ pypi ✓ bioconda

✓ conda-forge

Search any conda channel with channel:: prefix (eg. nvidia:: cuda)

Compatibility

☒ Show packages incompatible with selected architecture

Containers

No package added Search Conda and PyPI...

Recent Containers Edit environment.yml

Container settings: Docker linux/amd64 Get Container

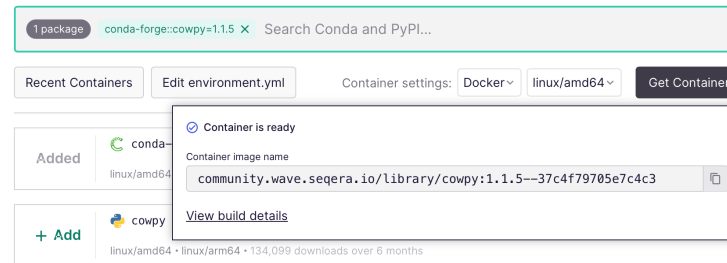
Add packages to fetch a container

nextflow - Containers - Docker

Imagem da Internet - Seqera

```
❶ training/hello-nextflow/solutions/4-hello-modules -> cowpy
bash: cowpy: command not found
❷ training/hello-nextflow/solutions/4-hello-modules -> █
```

Containers



```
❶ training/hello-nextflow/solutions/4-hello-modules -> cowpy
bash: cowpy: command not found
❷ training/hello-nextflow/solutions/4-hello-modules -> docker pull community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
1.1.5--37c4f79705e7c4c3: Pulling from library/cowpy
ca2678b20700: Pull complete
a192f22ebf5e: Pull complete
Digest: sha256:31d82e07d0f6198cbdb5b7160f8c47ea050ecc426ba2f9a70d253a3bddd318e
Status: Downloaded newer image for community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
❸ training/hello-nextflow/solutions/4-hello-modules -> █
```

nextflow - Containers - Docker

Imagem da Internet - Seqera

```
⊙ training/hello-nextflow/solutions/4-hello-modules -> cowpy
bash: cowpy: command not found
● training/hello-nextflow/solutions/4-hello-modules -> docker pull community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
1.1.5--37c4f79705e7c4c3: Pulling from library/cowpy
ca2678b20700: Pull complete
a192f22ebf5e: Pull complete
Digest: sha256:31d82e07d0f6198cbdb5b7160f8c47ea0500ecc426ba2f9a70d253a3bddd318e
Status: Downloaded newer image for community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
● training/hello-nextflow/solutions/4-hello-modules -> docker run --rm community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3 cowpy

< Cowacter, eyes:default, tongue:False, thoughts:False >

      ^ ^
      (oo)\_____
      (__) \       )\/\
           ||----w |
           ||     ||

⊙ training/hello-nextflow/solutions/4-hello-modules -> █

⊙ training/hello-nextflow/solutions/4-hello-modules -> docker run --rm -it community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3
root@8e54a22c42db:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@8e54a22c42db:/# █
```

nextflow - Containers - Docker

Imagem da Internet - Seqera

```
○ training/hello-nextflow/solutions/4-hello-modules -> docker run --rm -it community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3  
root@bc16aa5a236d:/# cowpy "Hello Antonio"
```

```
< Hello Antonio >
```

```
      ^ ^  
      (oo)\_____  
      (__) \       )\/\  
           ||----w |  
           ||     ||
```

```
root@bc16aa5a236d:/#
```

```
root@bc16aa5a236d:/# exit  
exit
```

```
○ training/hello-nextflow/solutions/4-hello-modules -> docker run --rm -it -v ../home/minhaAula community.wave.seqera.io/library/cowpy:1.1.5--37c4f79705e7c4c3  
root@53fa090aec34:/# cd /home/minhaAula/  
root@53fa090aec34:/home/minhaAula# ls  
hello-modules-2.nf  hello-modules-3.nf  hello-modules-4.nf  modules  
root@53fa090aec34:/home/minhaAula#
```

nextflow - Containers - Docker

Imagem da Internet - Seqera

```
hello-nextflow > solutions > 5-hello-containers > modules >  cowpy.nf >  process cowpy
1 // Generate ASCII art with cowpy (https://github.com/jeffbutters/cowpy)
2 process cowpy 
3
4
5
6   input:
7   path input_file
8   val character
9
10  output:
11  path "cowpy-${input_file}"
12
13  script:
14  """"
15  cat ${input_file} | cowpy -c "${character}" > cowpy-${input_file}
16  """"
17
18
```

Vai funcionar?

```
1 #!/usr/bin/env nextflow
2
3 // Include modules
4 include { sayHello } from './modules/sayHello.nf'
5 include { convertToUpper } from './modules/convertToUpper.nf'
6 include { collectGreetings } from './modules/collectGreetings.nf'
7 include { cowpy } from './modules/cowpy.nf'
8
9 /*
10  * Pipeline parameters
11  */
12 params {
13   input: Path = 'data/greetings.csv'
14   batch: String = 'batch'
15   character: String = 'turkey'
16 }
17
18 Preview DAG
19 workflow {
20
21   main:
22   // create a channel for inputs from a CSV file
23   greeting_ch = channel.fromPath(params.input)
24   .splitCsv()
25   .map { line -> line[0] }
26
27   // emit a greeting
28   sayHello(greeting_ch)
29   // convert the greeting to uppercase
30   convertToUpper(sayHello.out)
31   // collect all the greetings into one file
32   collectGreetings(convertToUpper.out.collect(), params.batch)
33   // generate ASCII art of the greetings with cowpy
34   cowpy(collectGreetings.out.outfile, params.character)
35
36   publish:
37   first_output = sayHello.out
38   uppercased = convertToUpper.out
39   collected = collectGreetings.out.outfile
40   batch_report = collectGreetings.out.report
41   cowpy_art = cowpy.out
42 }
43
44 output {
45   first_output {
46     path 'hello_containers/intermediates'
47     mode 'copy'
48   }
49   uppercased {
50     path 'hello_containers/intermediates'
51     mode 'copy'
52   }
53   collected {
54     path 'hello_containers/intermediates'
55     mode 'copy'
56   }
57   batch_report {
58     path 'hello_containers'
59     mode 'copy'
60   }
61   cowpy_art {
62     path 'hello_containers'
63     mode 'copy'
64   }
65 }
```

nextflow - Containers - Docker

Imagem da Internet - Seqera

```
hello-nextflow > solutions > 5-hello-containers > modules > X cowpy.nf > ...
1 // Generate ASCII art with cowpy (https://github.com/jeffbuttars/cowpy)
2 process cowpy {
3
4     container 'community.wave.seqera.io/library/cowpy:1.1.5--3db457ae1977a273'
5
6     input:
7     path input_file
8     val character
9
10    output:
11    path "cowpy-${input_file}"
12
13    script:
14    """
15    cat ${input_file} | cowpy -c "${character}" > cowpy-${input_file}
16    """
17 }
18
```

Vai funcionar?

```
1 #!/usr/bin/env nextflow
2
3 // Include modules
4 include { sayHello } from './modules/sayHello.nf'
5 include { convertToUpper } from './modules/convertToUpper.nf'
6 include { collectGreetings } from './modules/collectGreetings.nf'
7 include { cowpy } from './modules/cowpy.nf'
8
9 /*
10  * Pipeline parameters
11  */
12 params {
13     input: Path = 'data/greetings.csv'
14     batch: String = 'batch'
15     character: String = 'turkey'
16 }
17
18 Preview DAG
19 workflow {
20
21     main:
22     // create a channel for inputs from a CSV file
23     greeting_ch = channel.fromPath(params.input)
24                     .splitCsv()
25                     .map { line -> line[0] }
26
27     // emit a greeting
28     sayHello(greeting_ch)
29
30     // convert the greeting to uppercase
31     convertToUpper(sayHello.out)
32
33     // collect all the greetings into one file
34     collectGreetings(convertToUpper.out.collect(), params.batch)
35
36     // generate ASCII art of the greetings with cowpy
37     cowpy(collectGreetings.out.outfile, params.character)
38
39     publish:
40     first_output = sayHello.out
41     uppercased = convertToUpper.out
42     collected = collectGreetings.out.outfile
43     batch_report = collectGreetings.out.report
44     cowpy_art = cowpy.out
45 }
46
47 output {
48     first_output {
49         path 'hello_containers/intermediates'
50         mode 'copy'
51     }
52     uppercased {
53         path 'hello_containers/intermediates'
54         mode 'copy'
55     }
56     collected {
57         path 'hello_containers/intermediates'
58         mode 'copy'
59     }
60     batch_report {
61         path 'hello_containers'
62         mode 'copy'
63     }
64     cowpy_art {
65         path 'hello_containers'
66         mode 'copy'
67     }
68 }
```

nextflow - Containers - Docker

Imagem da Internet - Seqera

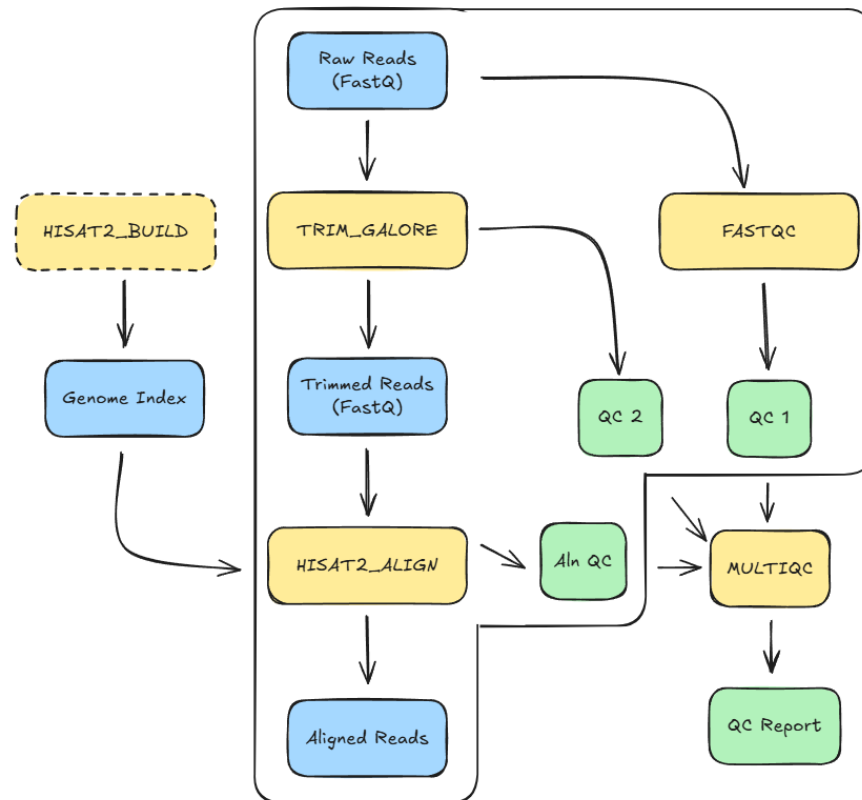
```
hello-nextflow > solutions > 5-hello-containers > modules > X cowpy.nf > ...
1 // Generate ASCII art with cowpy (https://github.com/jeffbuttars/cowpy)
2 process cowpy {
3
4     container 'community.wave.seqera.io/library/cowpy:1.1.5--3db457ae1977a273'
5
6     input:
7     path input_file
8     val character
9
10    output:
11    path "cowpy-${input_file}"
12
13    script:
14    """
15    cat ${input_file} | cowpy -c "${character}" > cowpy-${input_file}
16    """
17 }
18
```

Vai funcionar?

```
[Preview] README.md X X cowpy.nf X nextflow.config X
hello-nextflow > solutions > 5-hello-containers > nextflow.config
1 docker.enabled = true
2
```

```
1 #!/usr/bin/env nextflow
2
3 // Include modules
4 include { sayHello } from './modules/sayHello.nf'
5 include { convertToUpper } from './modules/convertToUpper.nf'
6 include { collectGreetings } from './modules/collectGreetings.nf'
7 include { cowpy } from './modules/cowpy.nf'
8
9 /*
10  * Pipeline parameters
11  */
12 params {
13     input: Path = 'data/greetings.csv'
14     batch: String = 'batch'
15     character: String = 'turkey'
16 }
17
18 Preview DAG
19 workflow {
20
21     main:
22     // create a channel for inputs from a CSV file
23     greeting_ch = channel.fromPath(params.input)
24                     .splitCsv()
25                     .map { line -> line[0] }
26
27     // emit a greeting
28     sayHello(greeting_ch)
29
30     // convert the greeting to uppercase
31     convertToUpper(sayHello.out)
32
33     // collect all the greetings into one file
34     collectGreetings(convertToUpper.out.collect(), params.batch)
35
36     // generate ASCII art of the greetings with cowpy
37     cowpy(collectGreetings.out.outfile, params.character)
38
39     publish:
40     first_output = sayHello.out
41     uppercased = convertToUpper.out
42     collected = collectGreetings.out.outfile
43     batch_report = collectGreetings.out.report
44     cowpy_art = cowpy.out
45 }
```

nextflow - Aula Prática





Obrigado!

antonio.rezende@fiocruz.br

